

Critical Reproduction and Stress-Test of a Machine-Learning Phishing-URL Detector

Data Science in Cyber — Final Project
Dr. Uri Itai

Source under evaluation:
shreyagopal / Phishing-Website-Detection-by-Machine-Learning-Techniques

Authors	Itay Cohen — 328316591 · Yarin Saraga — 330744392
Submission	Data Science in Cyber, 2026
Repository	https://github.com/itaychn137/project-phishing-critical-repro

1. Summary of the Source

Problem and its importance

The source addresses **phishing-website detection**: deciding, from a URL, whether the site it points to is a phishing attempt or a legitimate page. Phishing remains one of the most prevalent and cost-effective attack vectors in modern cybersecurity. It is the dominant entry point for credential theft and ransomware, scales trivially (a single kit can spawn thousands of URLs), and exploits human trust rather than software vulnerabilities, which makes purely technical defences insufficient. Automated, URL-level detection is attractive because it can act before a page is rendered and does not require inspecting potentially malicious page content.

Proposed solution

The author builds a supervised classifier over hand-engineered URL features. Seventeen features are extracted from each URL across three families — address-bar lexical features (presence of an IP address, the '@' symbol, URL length, path depth, redirection, an embedded 'https' token, URL-shortening services, and a prefix/suffix dash), host-based features (DNS record existence, web-traffic rank, domain age, and domain expiry), and HTML/JavaScript features (iframe usage, mouse-over status-bar manipulation, right-click disabling, and the number of web forwards). Several classifiers are trained and the best is selected.

Dataset

The modelling dataset combines two sources: 5,000 verified phishing URLs sampled from **PhishTank** (2020 snapshot) and 5,000 legitimate URLs sampled from the **University of New Brunswick (UNB) benign URL list**. After feature extraction the result is a table of 10,000 rows and 16 features plus a binary label (1 = phishing, 0 = legitimate), committed to the repository as `5.urldata.csv`.

Model and methodology

The author trains five models — Decision Tree, Random Forest, Multilayer Perceptron, XGBoost, and an autoencoder — on an 80/20 train/test split, and reports model quality using **test accuracy**. XGBoost is selected as the best model with a reported accuracy of **86.4%**. Our faithful reproduction obtains 0.871, confirming the headline number.

2. Critical Evaluation of the Author's Claims

The central claim of the source is implicit but clear: that the proposed feature set, paired with XGBoost, produces an **effective phishing detector**, evidenced by ~86% test accuracy. We were able to **reproduce the accuracy number exactly**, yet we argue that the number does **not support the underlying claim**. Three independent problems — each demonstrated empirically — show that the reported performance is largely an artefact of how the dataset was constructed and evaluated rather than evidence of phishing-detection capability.

Finding 1 — Cross-source leakage: the model learns the data source, not phishing

Positive (phishing) and negative (legitimate) examples were drawn from two *different* corpora. Whenever the classes differ systematically in some property unrelated to phishing, a classifier can exploit that property to separate the sources rather than the phenomenon. This is exactly what happens here. The XGBoost feature importance is overwhelmingly concentrated in a single feature, `URL_Length`, with an importance of **0.857** — the model is effectively a one-feature rule.

That feature is degenerate. Every legitimate URL in the dataset (100%) is flagged 'long' (≥ 54 characters), so a *short* URL is treated as a near-certain phishing signal. The reason is visible in the raw URLs, before any feature extraction: the UNB benign list is a corpus of long download/torrent URLs (median length **101** characters, $100\% \geq 54$), whereas PhishTank URLs are far shorter (median **57**, $54.5\% \geq 54$). `URL_Length` is therefore a proxy for 'which source did this come from', not for phishing. Real legitimate browsing URLs (e.g. `google.com`, `github.com`) are short and would be misclassified as phishing wholesale.

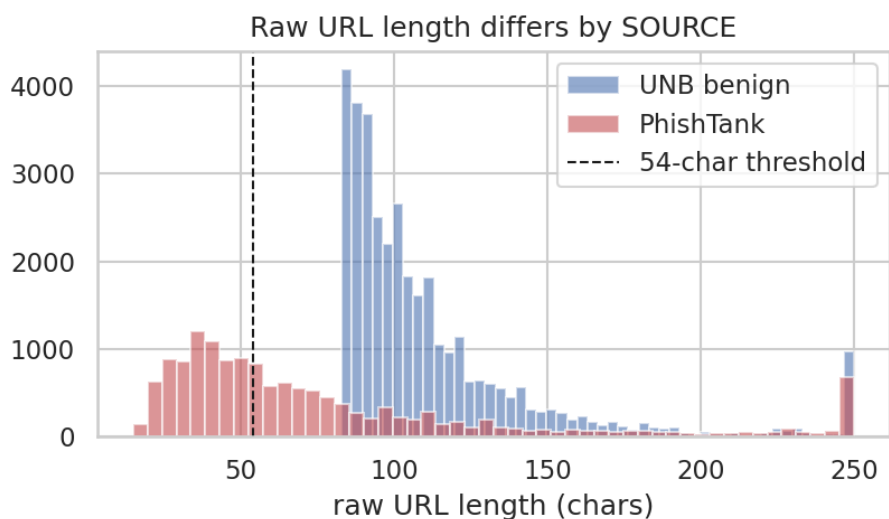


Figure 1. Raw URL length distributions by source. The two corpora differ sharply around the 54-character threshold, so `URL_Length` encodes the source.

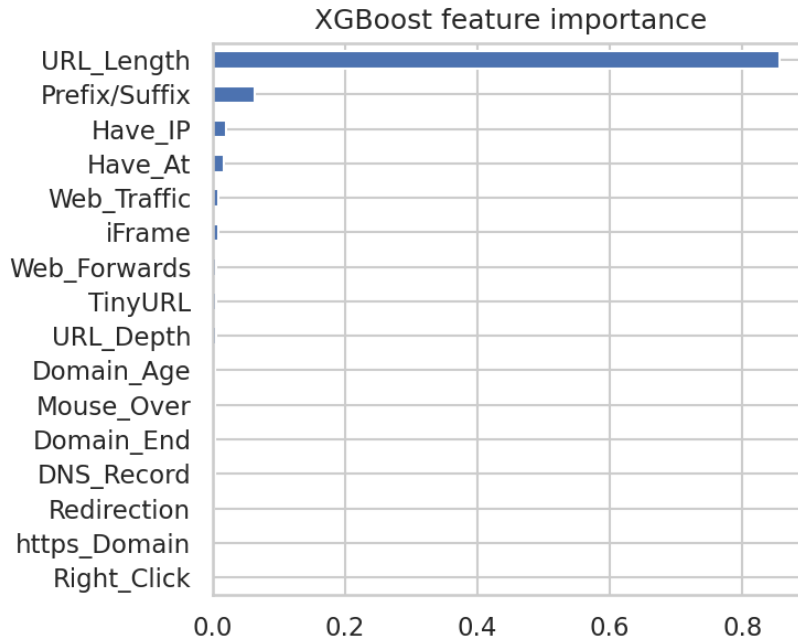


Figure 2. XGBoost feature importance. A single source-correlated feature dominates the model.

As confirmation, an XGBoost trained on only the two URL-shape features (URL_Length, URL_Depth) still reaches 0.774 accuracy and 0.818 ROC-AUC — most of the full model's performance comes from pure URL shape, the dimension on which the sources differ.

Finding 2 — Artificial class balance hides a precision collapse

The dataset is an exact 50/50 split, an artefact of sampling equal numbers from each source. In the real world phishing is a small minority of all URLs. Because accuracy and ROC-AUC are insensitive to this mismatch, they remain high while the operationally critical metric — precision — degrades severely. Holding the trained model fixed at its operating point (recall/TPR = 0.818, FPR = 0.074) and re-evaluating at realistic prevalence using the base-rate identity

$$precision(\pi) = \pi \cdot TPR / [\pi \cdot TPR + (1 - \pi) \cdot FPR]$$

yields the following:

Phishing prevalence	Precision	Recall
50%	0.917	0.818
10%	0.553	0.818
5%	0.369	0.818
1%	0.101	0.818

Precision falls from 0.917 at the synthetic 50/50 to 0.101 at a realistic ~1% prevalence, while recall is unchanged. At deployment the detector would raise roughly nine false alarms for every true catch — unusable in practice. The author never evaluates at a realistic base rate.

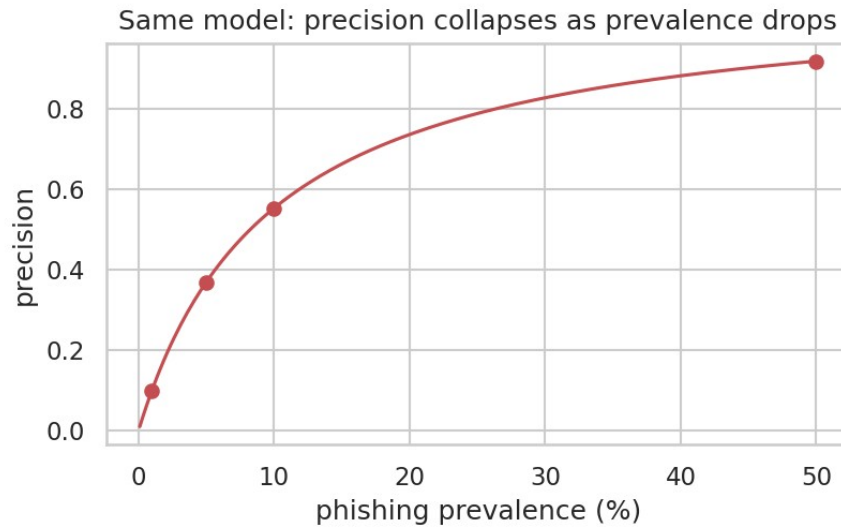


Figure 3. The same fixed model: precision collapses as phishing prevalence approaches real-world levels.

Finding 3 — Train/test contamination from a coarse feature space

With 15 binary flags and one small-integer feature, only a few hundred distinct feature vectors are representable, so the 10,000 rows duplicate heavily: only **880** unique (features+label) rows exist. A random 80/20 split therefore places identical points in both partitions — **96.5%** of test rows have an exact feature-vector twin in the training set. The model is partly graded on memorised points. De-duplicating and using a stratified split drops honest XGBoost accuracy from **0.871** to **0.733** — roughly 14 points of the reported score was contamination.

Worse, 109 of 771 distinct feature vectors (~14%) appear with *both* labels: the feature set physically cannot distinguish them, imposing a hard error floor the author does not acknowledge.

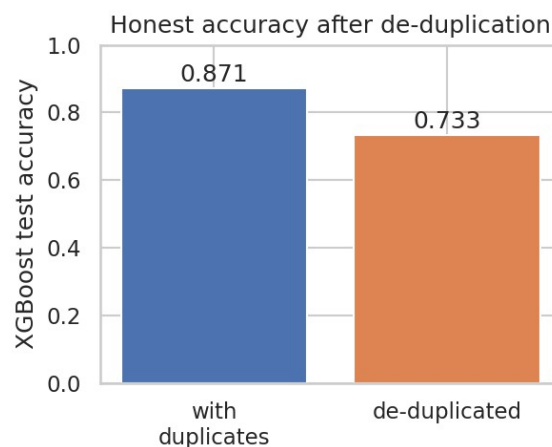


Figure 4. Removing duplicate feature vectors before splitting reveals substantially lower honest accuracy.

Is the methodology appropriate?

- **Metric choice.** Reporting accuracy alone on a balanced synthetic set is inappropriate for a rare-event security problem; precision, recall, $F\beta$, MCC and PR-AUC are needed.

- **Sampling.** Drawing positives and negatives from different sources introduces leakage; both classes should come from the same population.
- **Splitting.** A random split on duplicate-heavy data contaminates the test set; de-duplication or group-aware splitting is required.
- **No temporal validation.** Phishing drifts quickly, yet the time column is discarded, preventing train-on-past / test-on-future evaluation.
- **Model hygiene.** The MLP is trained on unscaled features (neural nets are scale-sensitive), and an autoencoder — an unsupervised anomaly-detection method — is applied to a fully labelled, balanced classification task, a conceptual mismatch.

Are the conclusions justified?

No. The accuracy figure is genuine and reproducible, but it reflects source identification, an unrealistic class balance, and test-set contamination rather than an ability to detect phishing. The claim of an effective detector is therefore not supported by the evidence. The work is, however, a valuable cautionary example of how leakage, base-rate neglect, and evaluation contamination can manufacture impressive-looking numbers.

3. Feature Engineering Analysis

Was feature engineering performed, and which features were used?

Yes — the source is almost entirely feature engineering. Raw URLs (and, for some features, live host/HTML lookups) are transformed into 17 features in three families: address-bar lexical features, host-based features, and HTML/JavaScript features (enumerated in Section 1). The dominant transformation is **binarisation**: nearly every signal is reduced to a 0/1 flag using a hard rule.

Transformations applied, why, and their effect

- **Thresholding.** Continuous quantities are cut to binary: URL_Length = 1 if length ≥ 54 ; domain age/expiry compared to fixed month thresholds; web-traffic rank compared to 100,000. Intended to simplify and de-noise, but it discards magnitude information and, for URL_Length, bakes in the source artefact of Finding 1.
- **Regular-expression matching.** URL-shortening services and prefix/suffix dashes are detected by regex. Reasonable and cheap, though the shortener list is static and ages.
- **External-lookup binarisation.** Web-traffic (Alexa rank) and WHOIS age/expiry are mapped to flags. These depend on third-party services and on the time of extraction (see Section 4).
- **No scaling, encoding, or dimensionality reduction.** Because features are already 0/1 there is no normalisation or categorical encoding. This is fine for tree models but harms the unscaled MLP.

Redundancy: how to spot it and how to tackle it

Two forms of redundancy are present. (i) **Feature redundancy**: the near-constant features https_Domain and Right_Click carry almost no variance. They are spotted via per-feature variance, the correlation heatmap, and near-zero feature importance, and tackled by dropping them — doing so leaves XGBoost accuracy unchanged, confirming they were dead weight. (ii) **Sample redundancy**: ~91% of rows are duplicates in feature space (Finding 3), spotted with a duplicate-row count and tackled by de-duplication before splitting.

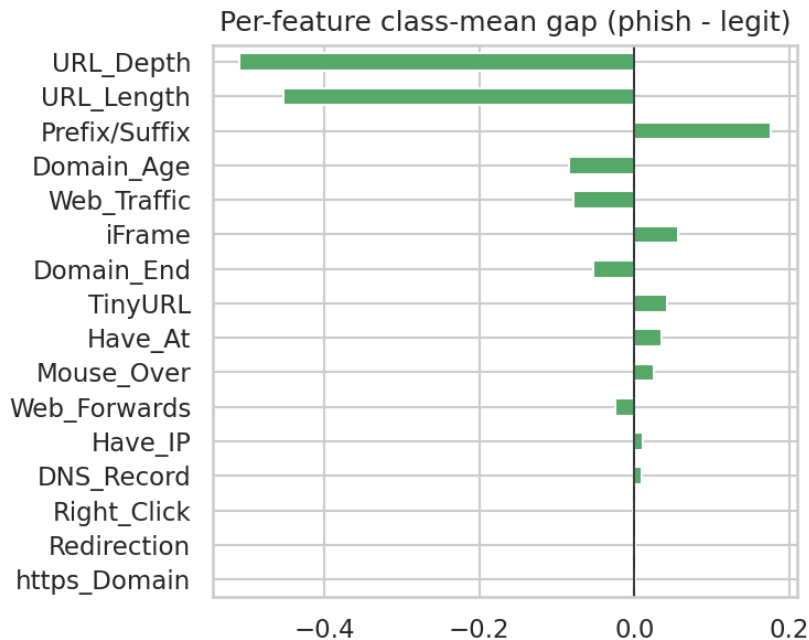


Figure 5. Per-feature class-mean gaps. Most features carry little class separation; the signal concentrates in a few URL-shape features.

Was the feature engineering meaningful? (mathematical and cybersecurity view)

Partly. Several features have sound cybersecurity rationale: an IP address in place of a domain, an '@' symbol that hides the true host, and prefix/suffix dashes are recognised phishing tells, and mathematically they do carry some class signal. The problem is that the single *most influential* feature, URL_Length, is also the least trustworthy: mathematically it is highly discriminative on this dataset, but that discriminative power comes from a source artefact, not from a property of phishing. Good feature engineering must be robust to how the data was collected; here the most important feature fails that test.

Additional features that could improve performance

- **Continuous URL length and token-level statistics** (length, number of dots/digits/special characters) instead of a single threshold flag.
- **Character n-gram / TF-IDF features** over the URL string, which capture obfuscation patterns without third-party services.
- **Lexical brand-impersonation cues** (edit distance to popular domains, homoglyph detection).
- **Consistent, reproducible host features** (TLS-certificate age and issuer, passive-DNS signals) drawn from a single source for both classes.
- **Time-aware features and a temporal split** to measure robustness to concept drift.
- **Most important of all: positives and negatives sampled from the same population**, so no feature can act as a source label.

4. Reproducibility Analysis

Can the code be executed, and are files and dependencies available?

The work splits cleanly into two halves. The **modelling half** is fully reproducible: the extracted feature table is committed to the repository, dependencies are standard (pandas, scikit-learn, xgboost), and re-running the pipeline reproduces the headline XGBoost accuracy (0.871 vs the reported 86.4%). The **feature-extraction half** does **not** reproduce today: it queries the Alexa rank API (discontinued by Amazon in May 2022), performs live WHOIS lookups (time-dependent and rate-limited), and fetches the live HTML of 2020 PhishTank URLs that are now offline.

Hidden preprocessing steps

Two implicit steps materially affect the data. First, when a host or HTML lookup fails, the extraction code **defaults the affected feature to the 'phishing' value (1)**. Because phishing URLs die quickly, 'could not reach this URL' becomes conflated with 'this is phishing', silently encoding reachability into the label. This explains why several HTML/JS features (e.g. Right_Click) are near-constant. Second, the down-sampling of 5,000 URLs from each larger source is neither seeded nor documented, so the exact sample is not recoverable.

Overall reproducibility

Mixed and, importantly, *honest to document*: the scientific result (the model and its metrics) can be reproduced from committed artefacts, but the data-generating pipeline cannot, and the hidden default-to-phishing rule biases the features. A reproduction that simply re-ran the committed CSV would inherit these flaws invisibly.

5. Experimental Results

Experiments performed and modifications introduced

We (i) faithfully reproduced the author's pipeline with fixed seeds; (ii) extended evaluation to a full metric suite rather than accuracy alone; and (iii) ran three diagnostic experiments — cross-source leakage, prevalence reweighting, and de-duplication — described in Section 2. All randomness is seeded (SEED = 12, matching the author) for reproducibility.

Models trained

Decision Tree (max_depth=5), Random Forest (max_depth=5), MLP (three hidden layers of 100), and XGBoost (learning_rate=0.4, max_depth=7), matching the author's configurations.

Evaluation metrics and why they matter for phishing

- **Accuracy** = $(TP+TN)/N$. Intuitive but misleading under class imbalance — the core failure in Finding 2.
- **Precision** = $TP/(TP+FP)$. 'Of the URLs we flagged, how many were truly phishing?' Low precision means many **false positives** — legitimate sites blocked, user friction, alert fatigue.
- **Recall** = $TP/(TP+FN)$. 'Of all real phishing, how much did we catch?' Low recall means **false negatives** — users exposed to live phishing, usually the costlier error.
- **F1 / F β** . Harmonic mean of precision and recall; F β with $\beta>1$ weights recall more, appropriate when a missed phish is worse than a false alarm.
- **MCC**. A balanced correlation using all four confusion cells; reliable under imbalance.
- **ROC-AUC vs PR-AUC**. ROC-AUC is optimistic under heavy imbalance; PR-AUC is the honest companion, so we report both.

Results

Model	Acc	Prec	Rec	F1	F2	MCC	ROC-AUC	PR-AUC
Decision Tree	0.807	0.970	0.643	0.773	0.689	0.657	0.850	0.861
Random Forest	0.814	0.985	0.644	0.779	0.692	0.672	0.887	0.919
MLP	0.862	0.949	0.770	0.850	0.800	0.738	0.920	0.941
XGBoost	0.871	0.921	0.818	0.866	0.837	0.747	0.925	0.945

Table 1. Full metric suite on the author's 80/20 split. XGBoost reproduces the reported ~86% accuracy; the experiments in Section 2 show why this number overstates real capability.

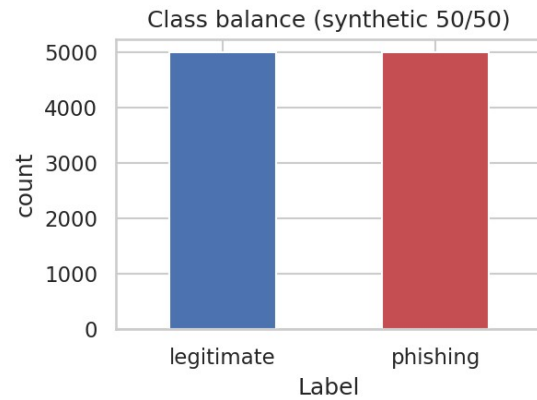


Figure 6. The synthetic 50/50 class balance.

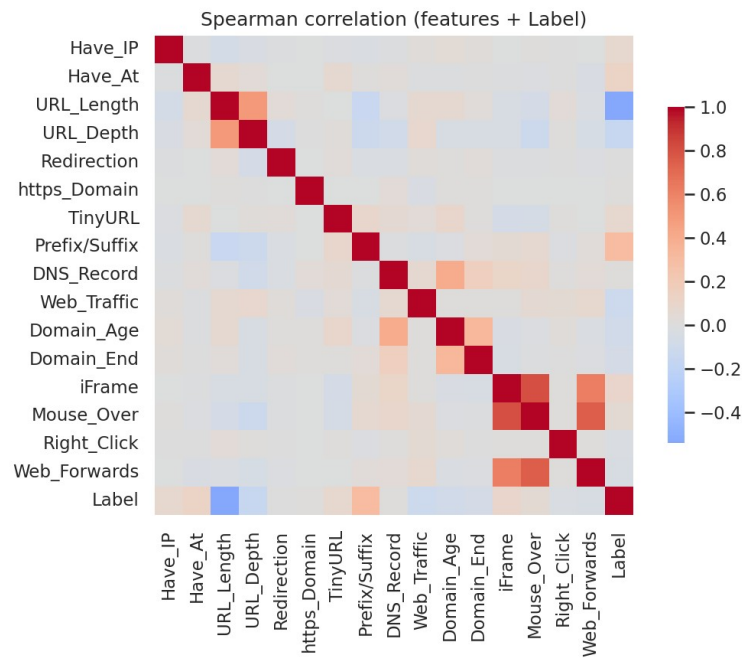


Figure 7. Spearman correlation. Spearman is used because most features are binary/ordinal, violating Pearson's linearity assumption; it coincides with the phi coefficient on binary pairs and correctly handles the ordinal URL_Depth.

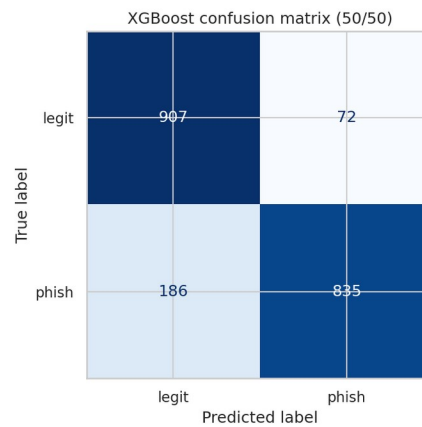


Figure 8. XGBoost confusion matrix on the balanced test set.

6. Error Analysis

False negatives (missed phishing) concentrate on phishing URLs that are short and structurally plain — precisely the URLs the model cannot distinguish from the long benign-source URLs. An attacker needs only a short, shallow URL to evade this detector. False positives are the more alarming deployment risk: because every legitimate URL in training is 'long', the entire short, real legitimate web looks phishing-shaped to the model. The contradictory feature vectors identified in Section 2 are an irreducible error source: identical features carry both labels, so no decision rule over this feature set can resolve them.

False-positive vs false-negative trade-off. In phishing detection a false negative typically carries the higher cost (a user is phished), which argues for tuning the decision threshold toward recall and reporting $F\beta$ with $\beta > 1$. However, raising recall on this model sharply increases false positives, and the precision collapse of Finding 2 means a recall-oriented threshold is operationally unviable at real prevalence. The trade-off cannot be fixed by thresholding; it requires fixing the data and features.

Conclusions

Key findings

- The reported ~86% accuracy is reproducible but does not reflect phishing-detection capability.
- The model is dominated by a single URL-shape feature that encodes the data source (cross-source leakage).
- The synthetic 50/50 balance hides a precision collapse from ~0.92 to ~0.10 at realistic prevalence.
- A coarse binary feature space duplicates ~91% of rows, contaminating the split; honest accuracy is ~0.73.

Strengths and weaknesses of the proposed solution

Strengths: interpretable, cheap, URL-only features; a committed feature table that makes the modelling half reproducible; an honest (not inflated by exotic tricks) modelling procedure. **Weaknesses:** cross-source leakage, artificial class balance, accuracy-only evaluation, train/test contamination, a non-reproducible and time-dependent extraction pipeline, and a hidden default-to-phishing bias.

Lessons learned and future improvements

- Sample positives and negatives from the same population; never let a feature double as a source label.
- Evaluate at realistic prevalence with precision, recall, $F\beta$, MCC and PR-AUC — not accuracy alone.
- De-duplicate or use group-aware splitting on coarse feature spaces.
- Preserve temporal structure and validate against concept drift.
- Replace third-party, time-dependent lookups with reproducible, content-independent features.

7. Executive Summary

This project reproduces and critically evaluates a published machine-learning phishing-URL detector that reports 86.4% XGBoost accuracy over 17 hand-engineered URL features. We confirm the accuracy is numerically reproducible from the committed feature table (our reproduction: 0.871), then show through three independent experiments that the figure does not support the claim of an effective detector.

First, the model is dominated by a single feature, URL_Length (importance 0.857), that encodes *which source* a URL came from rather than whether it is phishing: the legitimate corpus (UNB benign list, median length 101 characters) and the phishing corpus (PhishTank, median 57) differ sharply in raw URL length, and an XGBoost using only URL-shape features recovers most of the accuracy. Second, the dataset's synthetic 50/50 balance flatters every prevalence-insensitive metric; holding the model fixed and re-evaluating at a realistic ~1% phishing prevalence collapses precision from 0.917 to 0.101 while accuracy and ROC-AUC barely move. Third, the coarse binary feature space duplicates ~91% of rows so that 96.5% of test rows have an identical twin in training; de-duplicating drops honest accuracy to 0.733, and ~14% of distinct feature vectors carry both labels, imposing a hard error floor.

We also find the feature-extraction pipeline is no longer reproducible (it depends on the discontinued Alexa API, live WHOIS, and live fetches of dead URLs) and contains a hidden default-to-phishing rule on lookup failure that biases the features. We conclude that the author's impressive accuracy is an artefact of dataset construction and evaluation choices. We do **not** recommend the pipeline as a phishing detector, but it is an excellent, concrete case study in leakage, base-rate neglect, and evaluation contamination — the failure modes this course emphasises.

8. Summing It Up

Problem: Classifying URLs as phishing or legitimate before the page is visited.

Selected source: shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques (GitHub tutorial-style project).

Dataset: 5,000 PhishTank phishing URLs + 5,000 UNB benign URLs, reduced to 16 binary/ordinal features (10,000 rows).

Methodology employed: Hand-engineered URL features → 80/20 split → Decision Tree, Random Forest, MLP, XGBoost, autoencoder → evaluation by accuracy only.

Main findings of our reproduction: The headline accuracy reproduces (~ 0.87), but it is driven by cross-source leakage, inflated by an artificial 50/50 balance, and contaminated by train/test duplication; honest de-duplicated accuracy is ~ 0.73 and real-prevalence precision is ~ 0.10 .

Were the author's claims supported: No. The accuracy is real but does not evidence phishing-detection capability; the methodology is inappropriate for a rare-event security task.

Most important insights: Positives and negatives must come from the same population; rare-event problems must be evaluated at realistic prevalence with precision/recall/PR-AUC; coarse feature spaces require de-duplication before splitting.

Do we recommend this project for similar problems: Not as a detector or a template for building one. Yes — as a teaching example of how leakage and evaluation choices manufacture misleading results.

Final conclusion: A faithful, honest reproduction shows the source's results are an artefact of dataset construction and evaluation rather than evidence of a working phishing detector. The critical value of the project lies in identifying precisely why a reproducible 86% accuracy is, in this case, not trustworthy.

Reproducibility note: all quantitative results were produced from the committed dataset 5.urldata.csv with a fixed seed (SEED = 12). The accompanying notebook regenerates every figure and number in this report.